Mastering Augmented Reality with Unity

Prof. Jooho Cha, Ph.D.
jhcha@chungwoon.ac.kr

Dept. of Multimedia Science
Chungwoon University

Chapter 5. Creating Your First Custom Component

- Introduction
  - We will take a quick peek at C# within Unity, and see how we can use it to create a custom component.
  - We will also see how to communicate with other components and manipulate their values using C#.
  - If you are very new to programming and C#, it is recommended to go over the C# tutorials: https://docs.microsoft.com/en-us/dotnet/csharp/

Chapter 5. Creating Your First Custom Component

- Structure
  - Introduction to C# in Unity
  - Exploring the **MonoBehaviour**
  - Serialized variables
  - Debug class of Unity
  - Time in Unity
  - **GetComponent()**
  - Building a simple 3D clock

Chapter 5. Creating Your First Custom Component3D

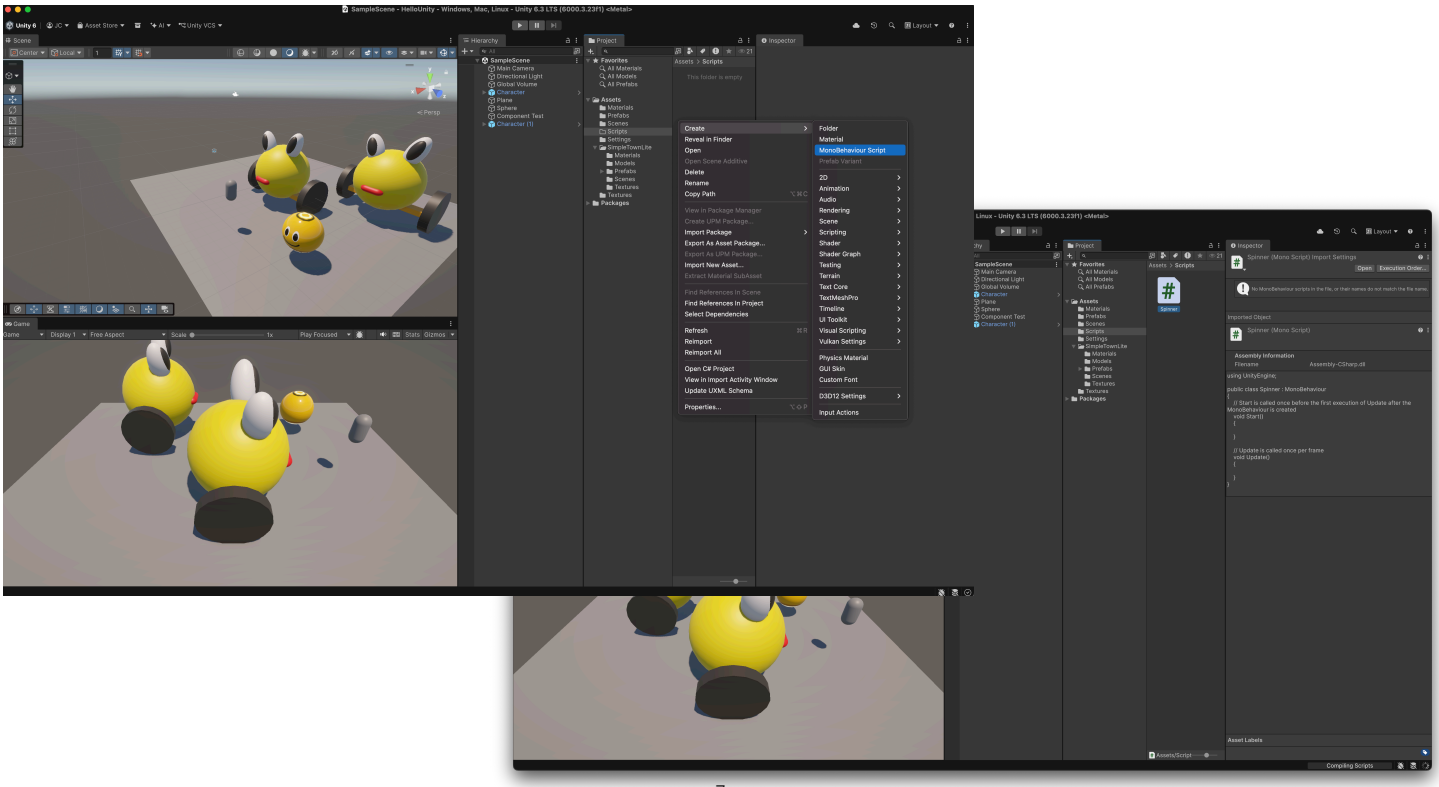
- Objectives
  - We will be able to understand followings:
    - How C# scripts work with Unity
    - The core features of C# syntax with Unity
    - How to create custom components with C# language, and use those components to create a simple 3D clock

Introduction to C# in Unity

- In Unity, a C# script is considered as a component.
- A basic component script is a class file derived from **MonoBehaviour** class created by Unity.
  - Unity generates a class inherited from the **MonoBehaviour** class.
  - A C# script works like a blueprint for a custom component that can be attached to any game object in the scene.

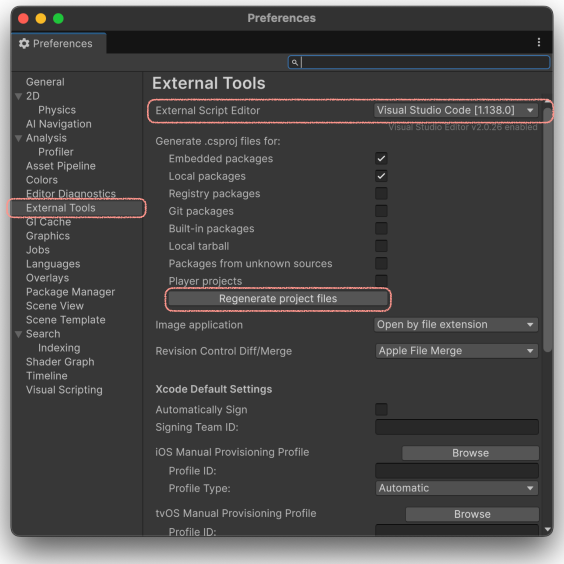
Introduction to C# in Unity

- Let us create our first component by following the given steps:
  1. Go to our SampleScene with the character and the smiley face sphere.
  2. Create a new folder called **Scripts** in the **Assets** folder.
  3. Inside the **Scripts** folder, right-click and click **Create** | **MonoBehaviour Script**.
  4. This will generate a default script called **NewMonoBehaviourScript**. We have to immediately rename it to **Spinner**.



Introduction to C# in Unity

5. Make sure you have selected Visual Studio (Code) as the IDE for C# in Unity.
6. Go to **Edit** | **Preferences** (**Unity** | **Settings...** on Mac).
7. Select **External Tools** from the **Preferences** window.
8. For the **External Script Editor**, choose the Visual Studio (Code).
9. If you do not see the editor in the list, you can browse and choose.



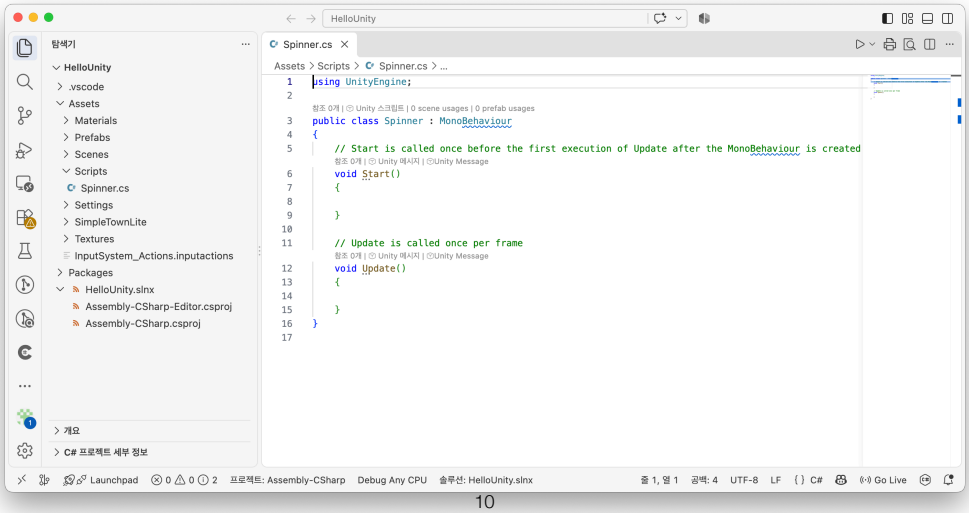
Introduction to C# in Unity

10. Click on the **Regenerate profile files** button. Now, the Unity editor is configured with Visual Studio (Code). This is only a one-time process.

11. Close the window, go to the **Scripts** folder under the **Assets**, and double-click on the script (**Spinner**) to launch it through Visual Studio (Code).

Exploring the MonoBehaviour

- As soon as you open the new C# script, this will be the template code you get:



Exploring the MonoBehaviour

- In Unity, the script must be attached to a game object to execute it.
  - The class name must be the same as the filename and is inherited from the **MonoBehaviour** component.
- There are two primary methods in any C# template script:
  - **Start()**: This method always executes only once at the very moment the game is started. Usually, it is used to initialize things in the scripts.
  - **Update()**: This is a loop method and executes once every frame. It can be used for any continuous checks, input detection, movement of objects, and so on.

Serialized variables

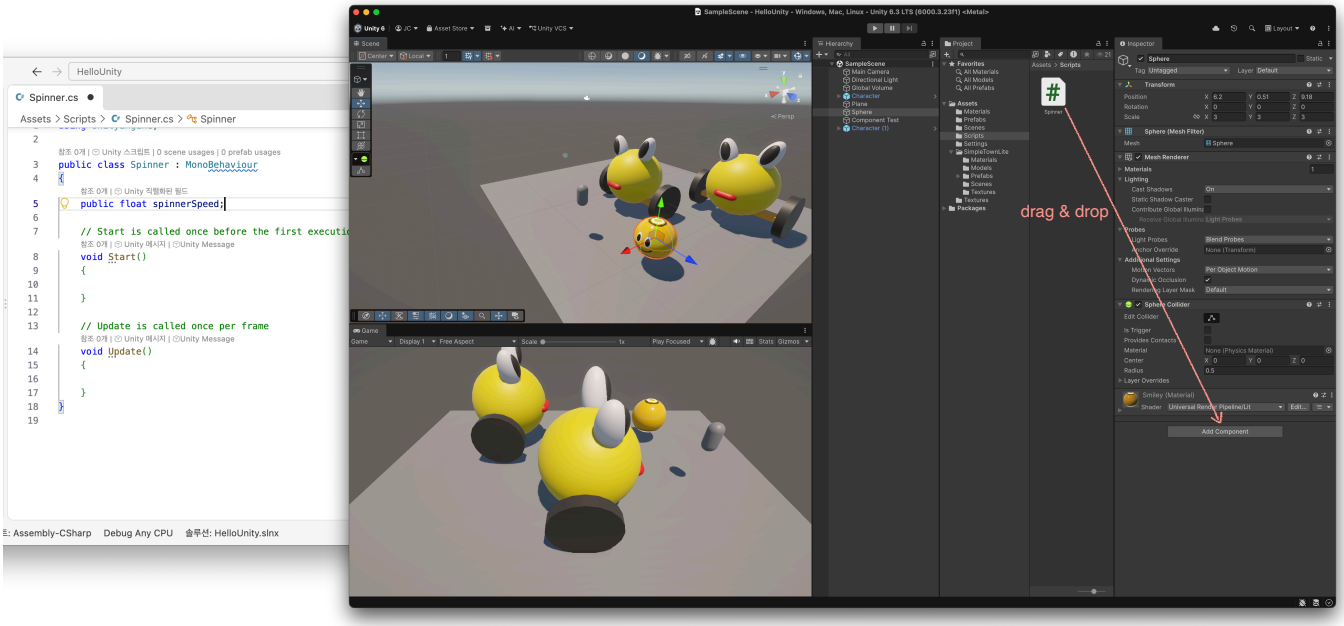
Serialized Variable(직렬화된 변수) ➜ C# 스크립트의 변수 값을 Unity Editor의 **Inspector 창에서 보고 수정할 수 있도록 만든 변수**

- Follow the given steps for serialized variables:
  1. Before utilizing the **Start()** method, create a simple **public float** variable named as **spinnerSpeed** as follows:

     public float spinnerSpeed;

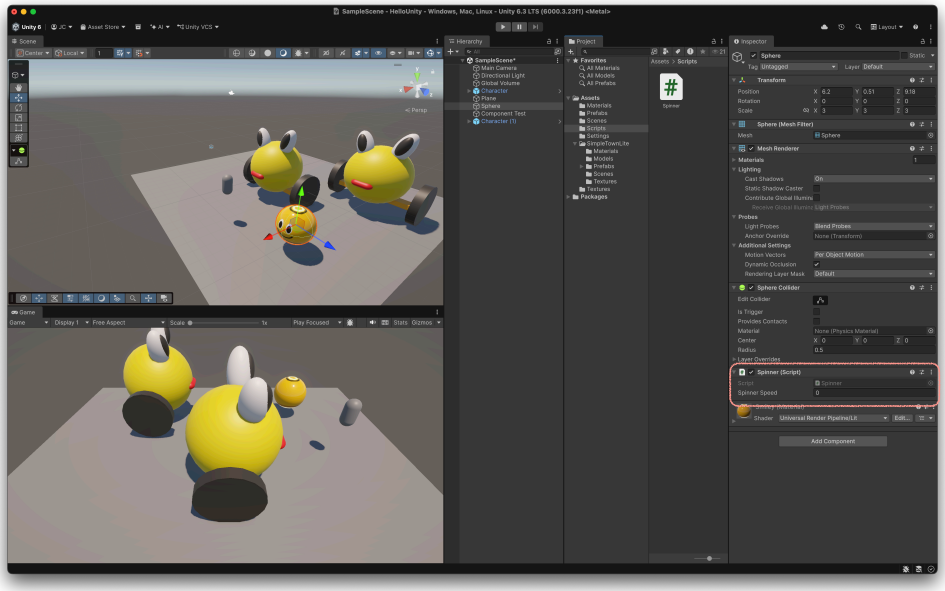
  2. Now save the script and go back to Unity.
  3. Select the smiley **Sphere** object we created before.
  4. Drag and drop the **Spinner** script to the **Sphere** object.

Serialized variables



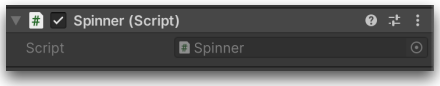
Serialized variables

5. Now you may see that a new custom component **Spinner** got attached to the **Sphere**, along with a text field to input the **Spinner Speed** value.



Serialized variables

6. Go back to the code, change **public** to **private**, save the script, and go back to Unity.
7. Select the **Sphere** and notice that the variable input is gone. This is because the access modifier is **private**.



8. We can use the **[SerializeField]** attribute in order to ask Unity to serialize it. This allows the Unity interface to show and modify the value. However, being a private variable, other scripts do not have the permission to the variable directly.

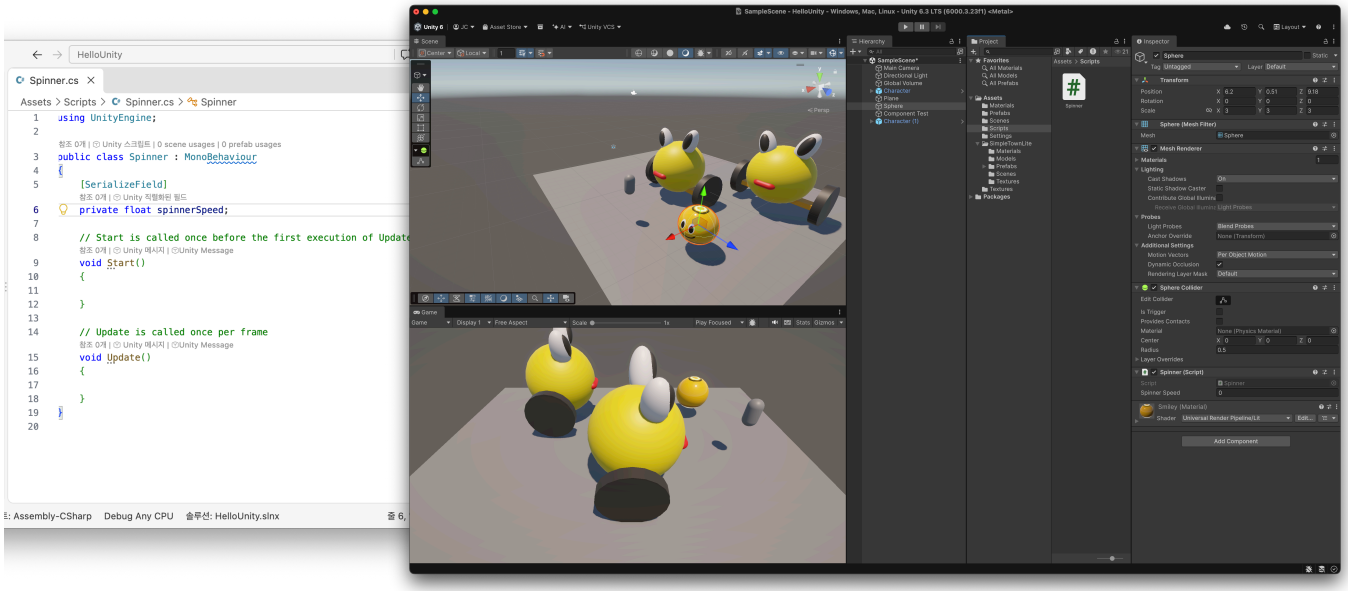
Serialized variables

9. Change the variable to a serialized field by adding the **[SerializeField]** attribute:

   [SerializeField] private float spinnerSpeed;

10. Save the script and go back to Unity. Notice that the field for **spinnerSpeed** is now back.

Serialized variables



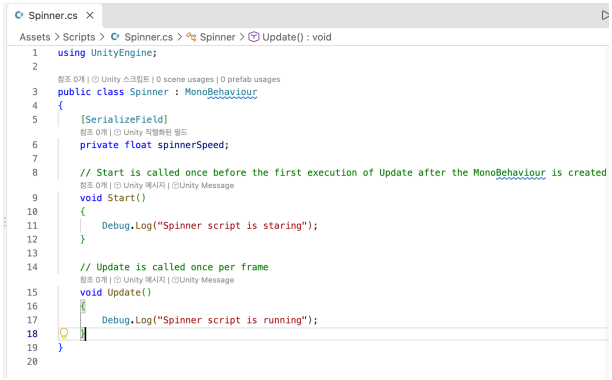
Debug class of Unity

- In Unity, you can use the **Debug** class to print messages in Unity console.
- The most used method of the **Debug** class is the **Log()** method.
- Open the **Spinner** script, and add the following line into the **Start()** method:

  Debug.Log("Spinner script is starting");

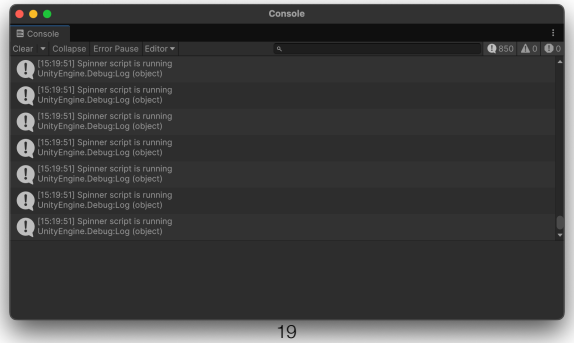
- Add this to the **Update()** method:

  Debug.Log("Spinner script is running");



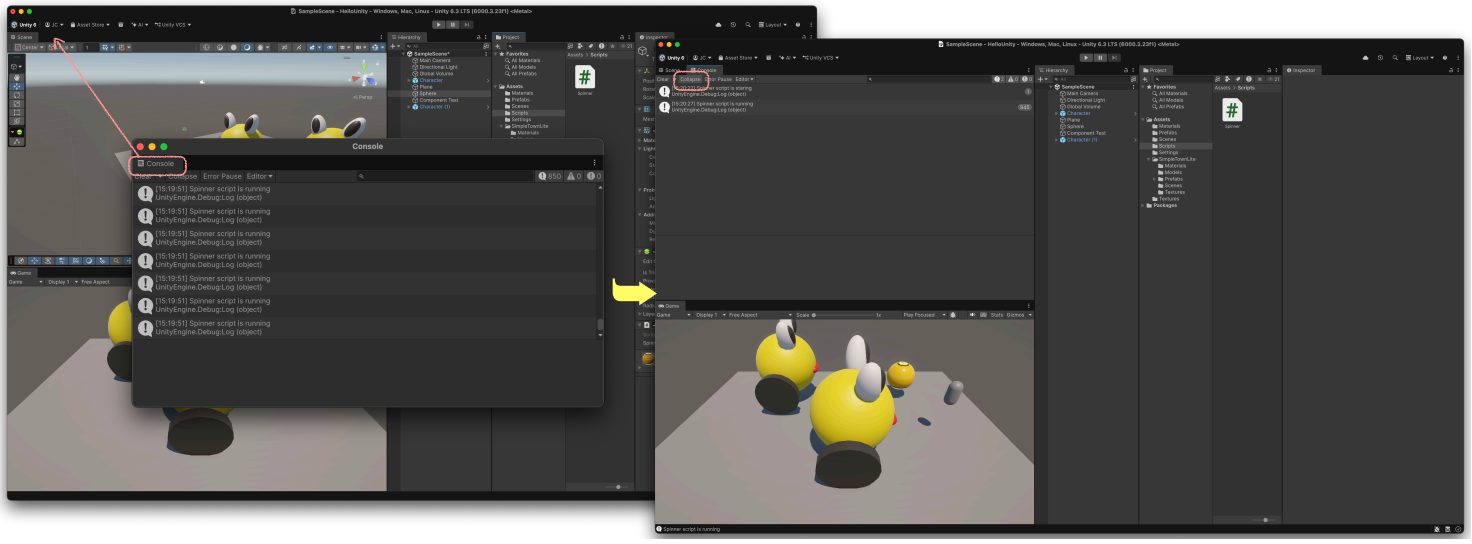
Debug class of Unity

- Save the script and run the game.
- Open the console window (**Window** | **General** | **Console**) and see how it prints **Spinner script is starting!** at once and **Spinner script is running!** infinitely (until you stop playing):



Debug class of Unity

- Drag the Console window to move it next to the Scene window as we will be using it often for debugging.



Time in Unity

- Unity uses the clock speed of the computer in order to perform frame-based calculations.
- Also, Unity uses the internal clock to track elapsed time through the **Time** module.
- **Update()** method is called once per frame while the Unity application is running.
  - The frame rate can vary depending on the workload of the CPU, the complexity of the application, and the resources available to Unity.

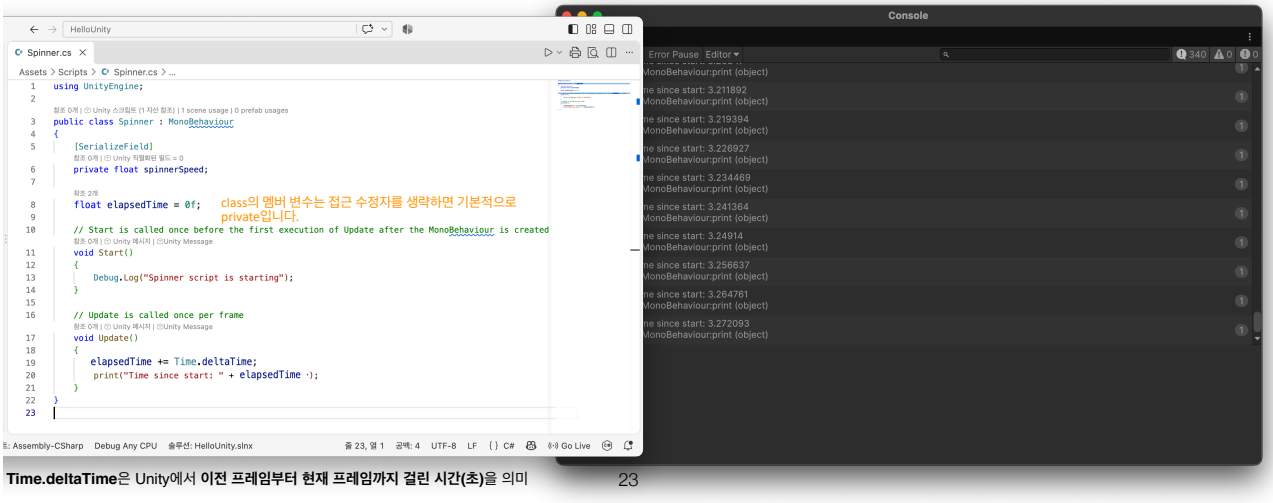
Time in Unity

Unity의 **Time** 클래스는 게임이 실행되는 동안 시간과 관련된 정보를 제공
- **Time.time**: 게임이 시작된 후 경과한 게임 시간(초)
- **Time.deltaTime**: 이전 프레임부터 현재 프레임까지 경과한 시간(초)

- So, how do we calculate actual elapsed time in the **Update()** method?
  - This can be achieved by using a variable provided by Unity called **Time.deltaTime**.
  - **Time.deltaTime** is a **float** value that represents the time, in seconds, that has elapsed since the previous frame.
  - Therefore, if we continuously add **Time.deltaTime** to a variable that is initially set to **0** in the **Update()** method, the accumulated value represents the total elapsed time since the game started.

Time in Unity

- Change the contents of the **Sphere** class with the following code, save it, and run the game.



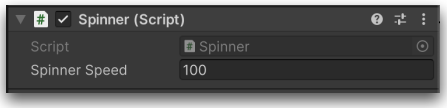
Time.deltaTime은 Unity에서 **이전 프레임부터 현재 프레임까지 걸린 시간(초)**을 의미

GetComponent( )

- The **GetComponent()** method is used to access other components attached to the same GameObject as the script.
- By default, Unity provides direct access to the **gameObject** and **transform** properties of the GameObject that the script is attached to.
- However, to access other component, we need to use **GetComponent<T>()**.
  - Here, the **T** represents the **type of component** we want to access, which is the class name of the component.

GetComponent( )

- Because the **GetComponent()** method can be relatively expensive, it is a good practice to call it once in the **Start()** method and cache the result for later use.
- Modify the **Spinner** script as shown on the right, change **spinnerSpeed** to 100 in the Inspector window, and press **Play** to test it:



Spinner.cs

Assets > Scripts > Spinner.cs > ...

```
sing UnityEngine;

참조 0개 | Unity 스크립트 (1 자산 참조) | 1 scene usage | 0 prefab usages
blic class Spinner : MonoBehaviour

    [SerializeField]
    참조 1개 | Unity 직렬화된 필드 = 100
    private float spinnerSpeed;

    참조 3개
    float elapsedTime = 0f;
    참조 3개
    private MeshRenderer meshRenderer;

    // Start is called once before the first execution of Update after the MonoBehaviour i
    참조 0개 | Unity 메시지 | Unity Message
    void Start()
    {
        meshRenderer = GetComponent<MeshRenderer>();
        if (meshRenderer == null)
        {
            Debug.LogError("MeshRenderer component not found on the GameObject.");
        }
    }

    // Update is called once per frame
    참조 0개 | Unity 메시지 | Unity Message
    void Update()
    {
        transform.Rotate(0, spinnerSpeed * Time.deltaTime, 0);
        elapsedTime += Time.deltaTime;
        if(elapsedTime >= 0f)
        {
            elapsedTime = 0f;
            meshRenderer.material.color = Random.ColorHSV();
        }
    }

```

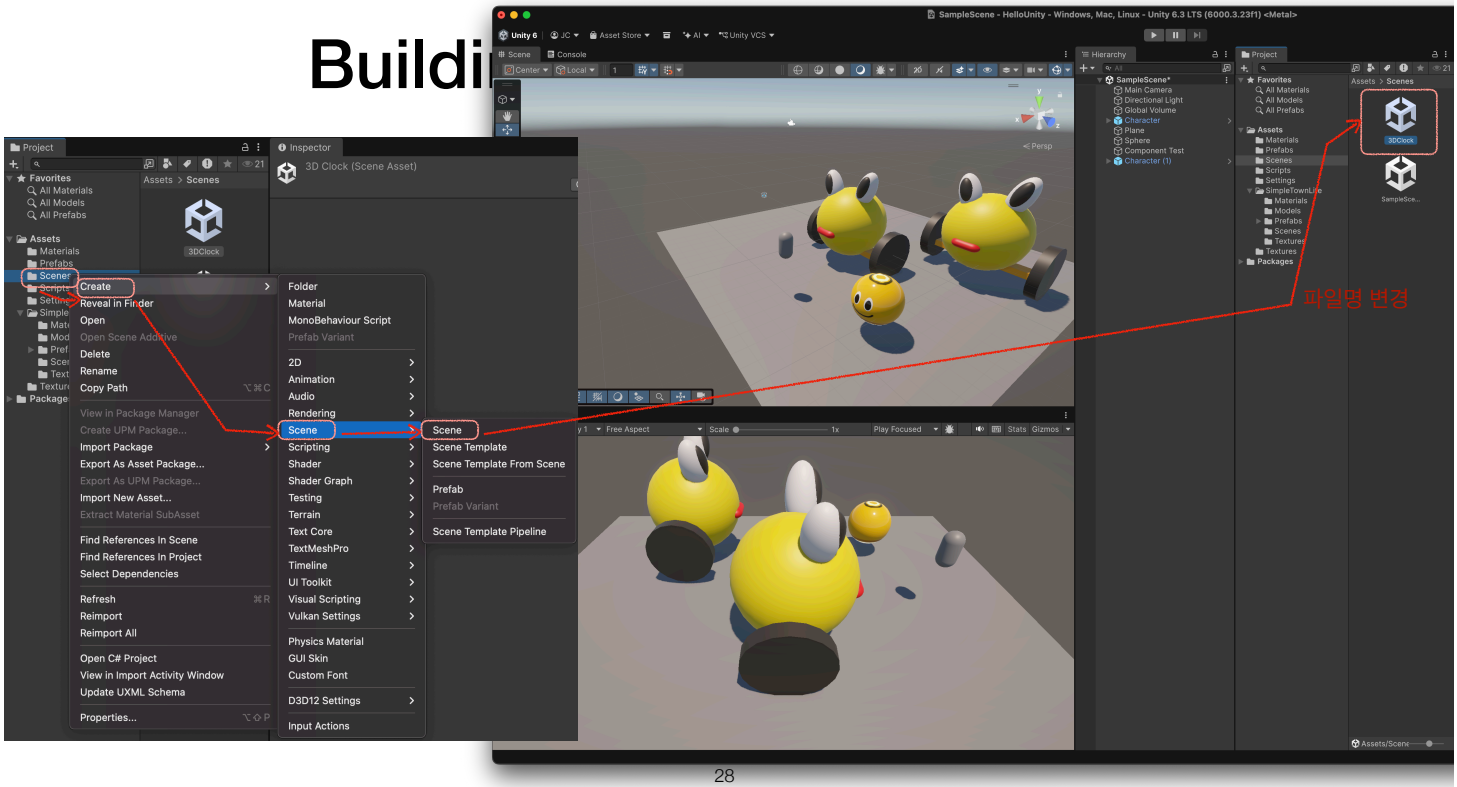
ssembly-CSharp Debug Any CPU 솔루션: HelloUnity.slnx 줄 33, 열 1 공백: 4 UTF-8 LF { } C#

GetComponent( )

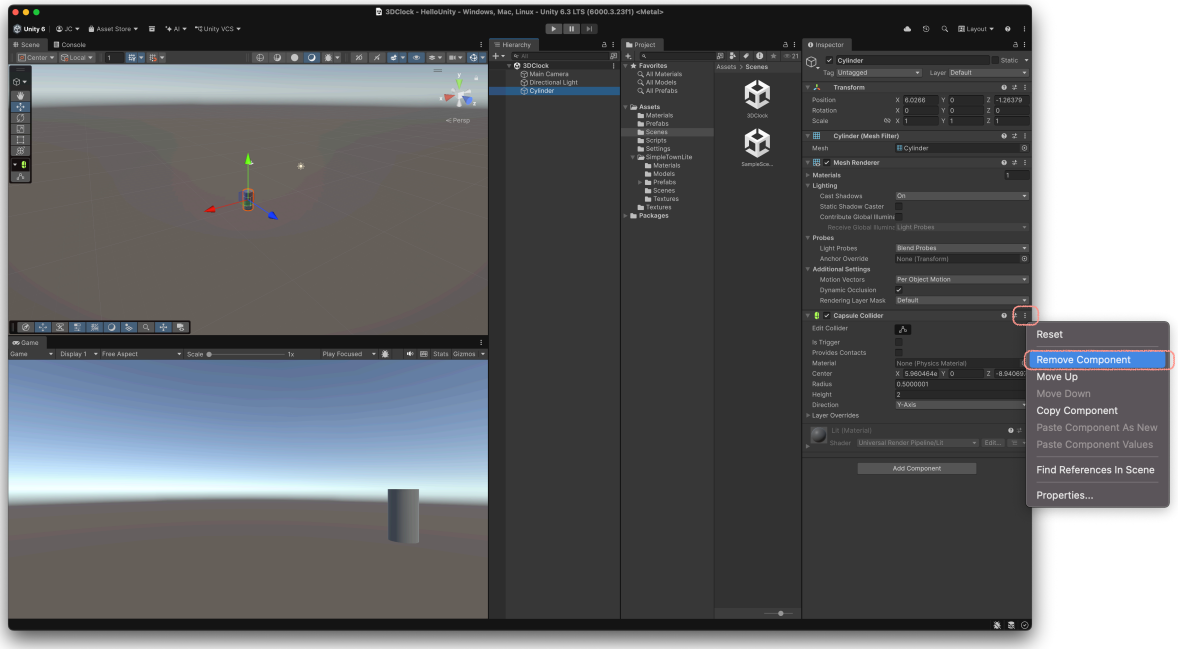
Building a simple 3D clock

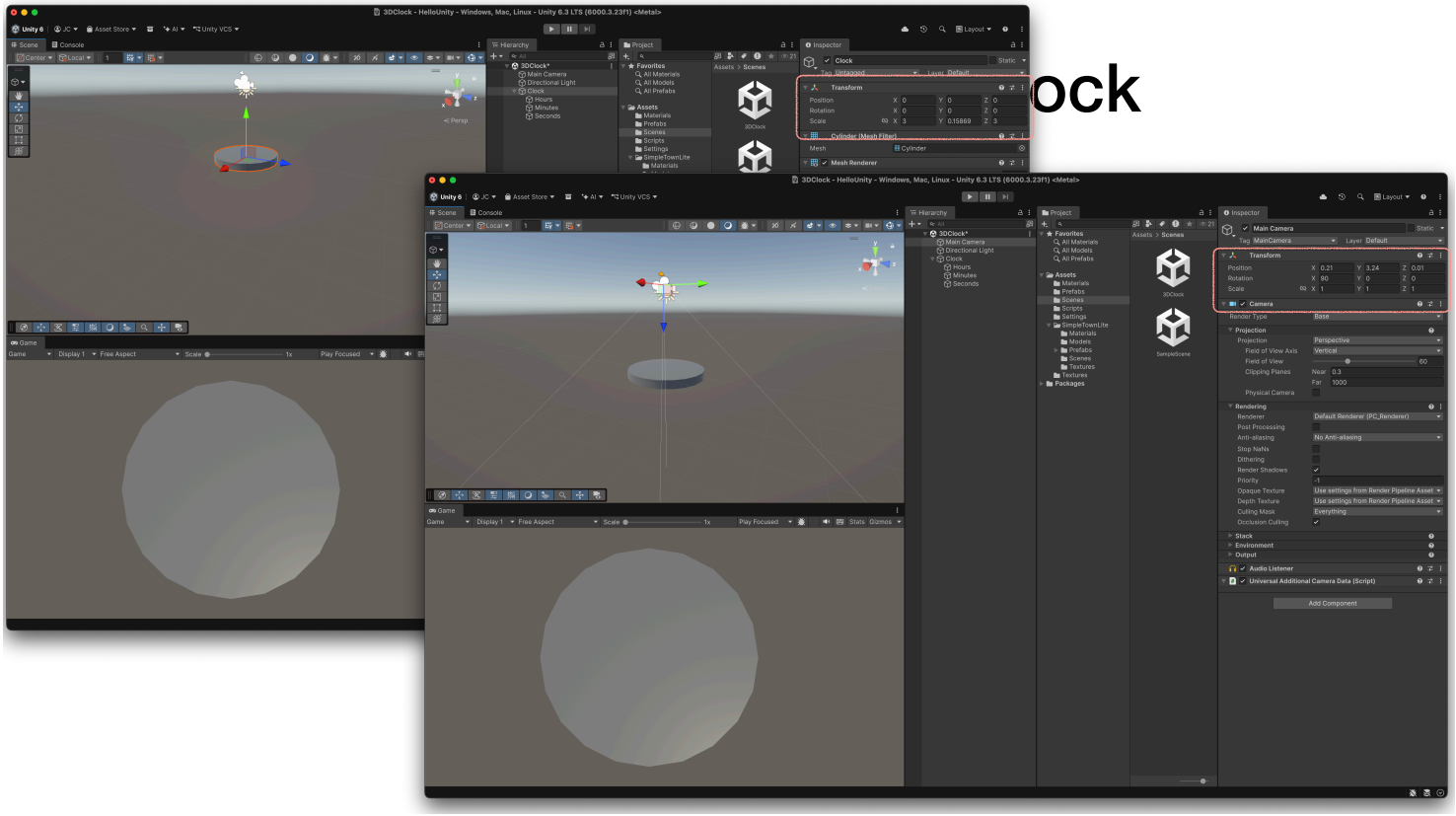
- Now, let's try to build a simple 3D clock. Follow the steps below:
  1. Create a new **Scene** in the **Scenes** folder and rename it **3DClock**.
  2. Open the scene and create a **Cylinder** 3D object. Remove the **Capsule Collider** component by clicking the three-dot (⋮) menu in the upper-right corner of the component.
  3. Scale the **Cylinder** so that it looks like a clock face, and rename it **Clock**.

Buildi



Building a simple 3D clock

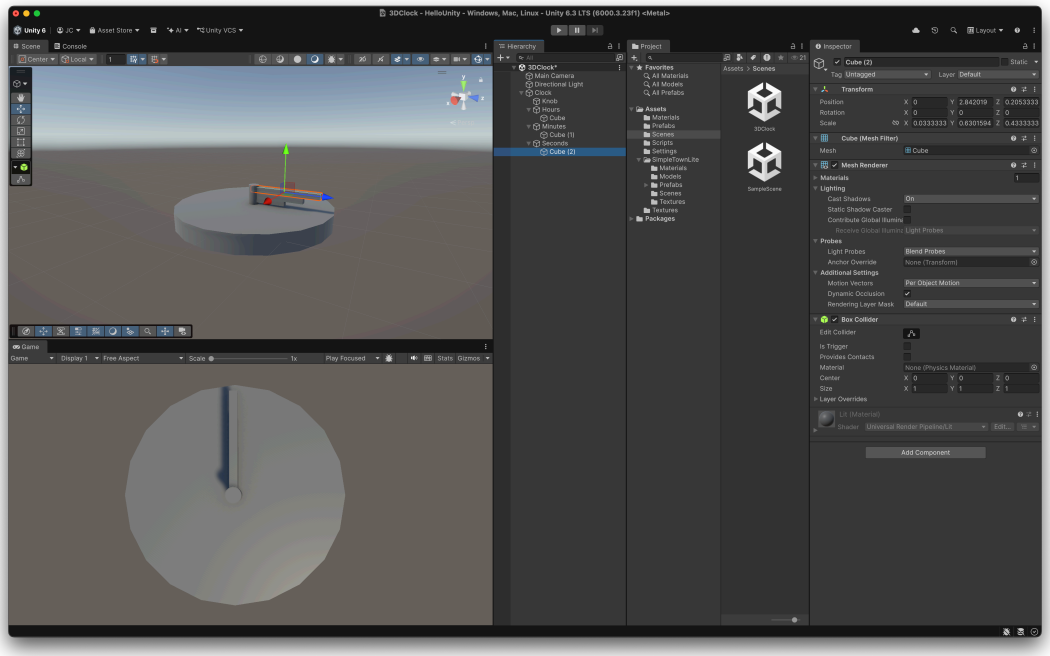




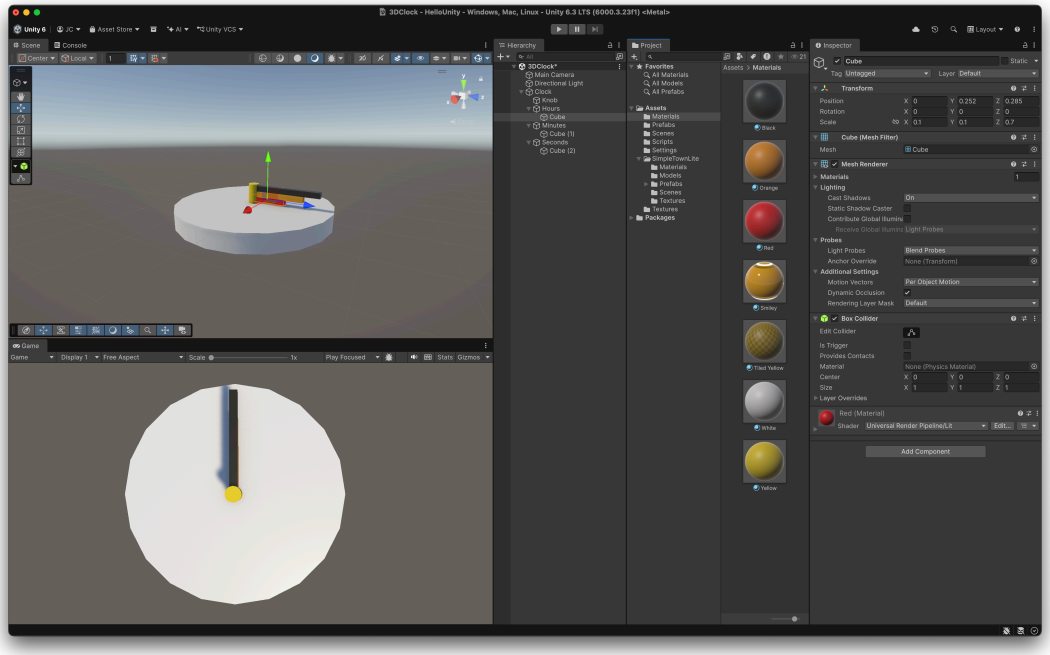
Building a simple 3D clock

4. Inside the **Clock** GameObject, create three GameObjects and rename them **Hours**, **Minutes**, and **Seconds**.
5. Add another **Cylinder**, move it to the center of the main clock face, and rename it **Knob**.
6. Create three **Cubes** and scale them so that they look like clock hands. Move each **Cube** into the corresponding **Hours**, **Minutes**, and **Seconds** GameObject.
7. You can use different materials to give each **Cube** a different color.

Building a simple 3D clock

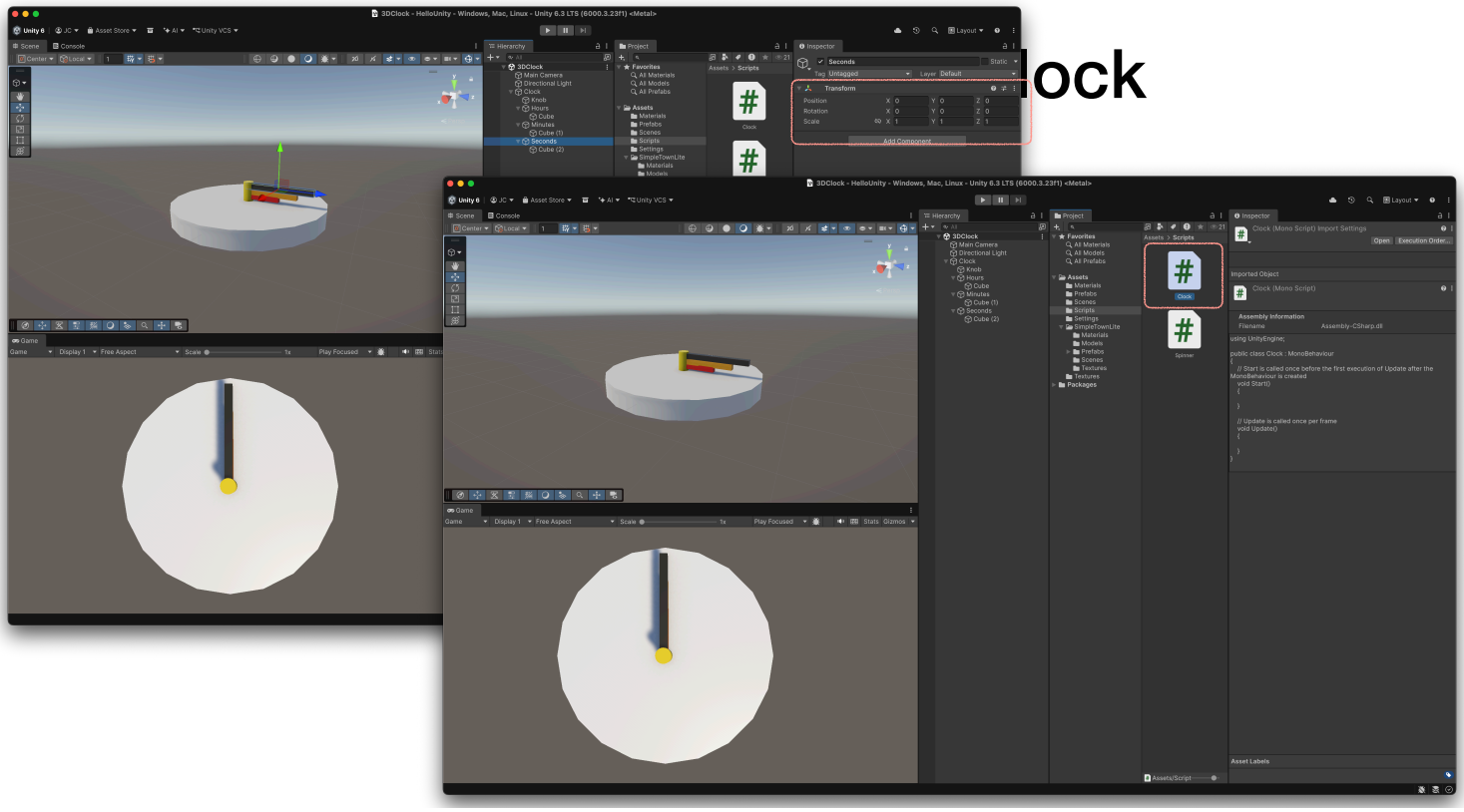


Building a simple 3D clock

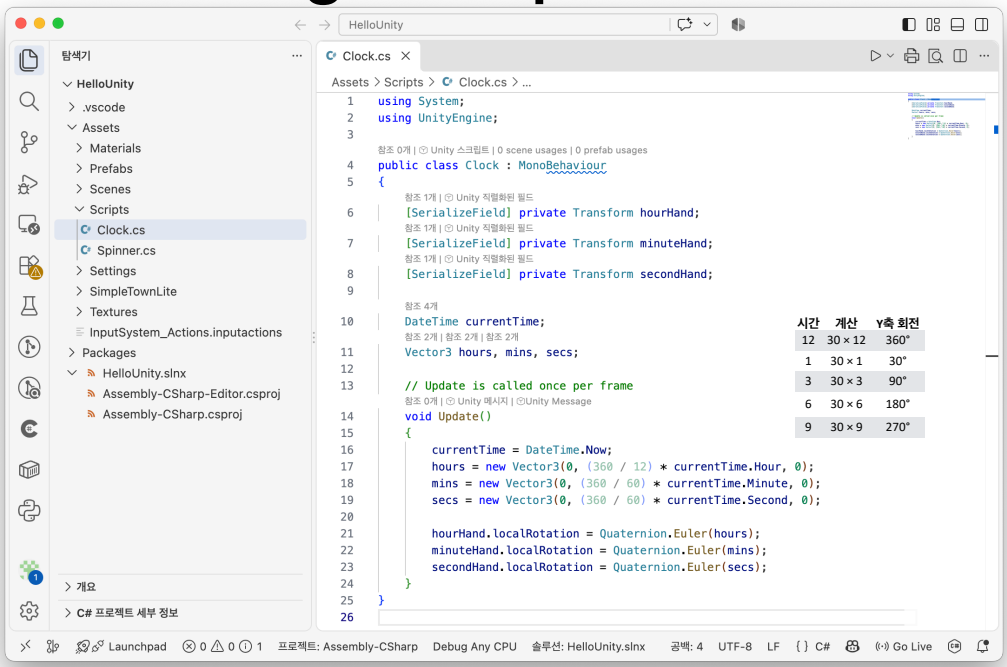


Building a simple 3D clock

8. Finally, you should have a clock model with the time set to **12:00:00**. The **Hours**, **Minutes**, **Seconds**, and **Knob** GameObjects should all have a position of (**0,0,0**) and a rotation of (**0,0,0**). If you select any of the clock hand GameObject and rotate it around the **Y-axis**, you can see it move around the clock face.
9. Save the scene.
10. Go to the **Scripts** folder and create a new **Monobehaviour Script**. Rename it as **Clock**.
11. Open the script and modify it as follows:



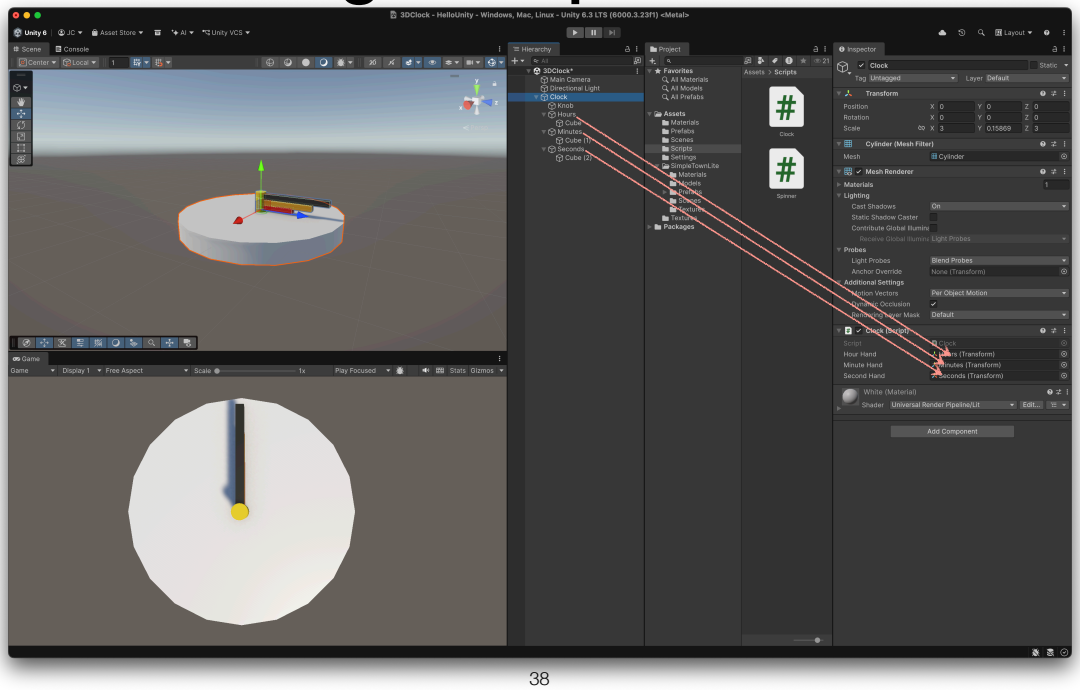
Building a simple 3D clock

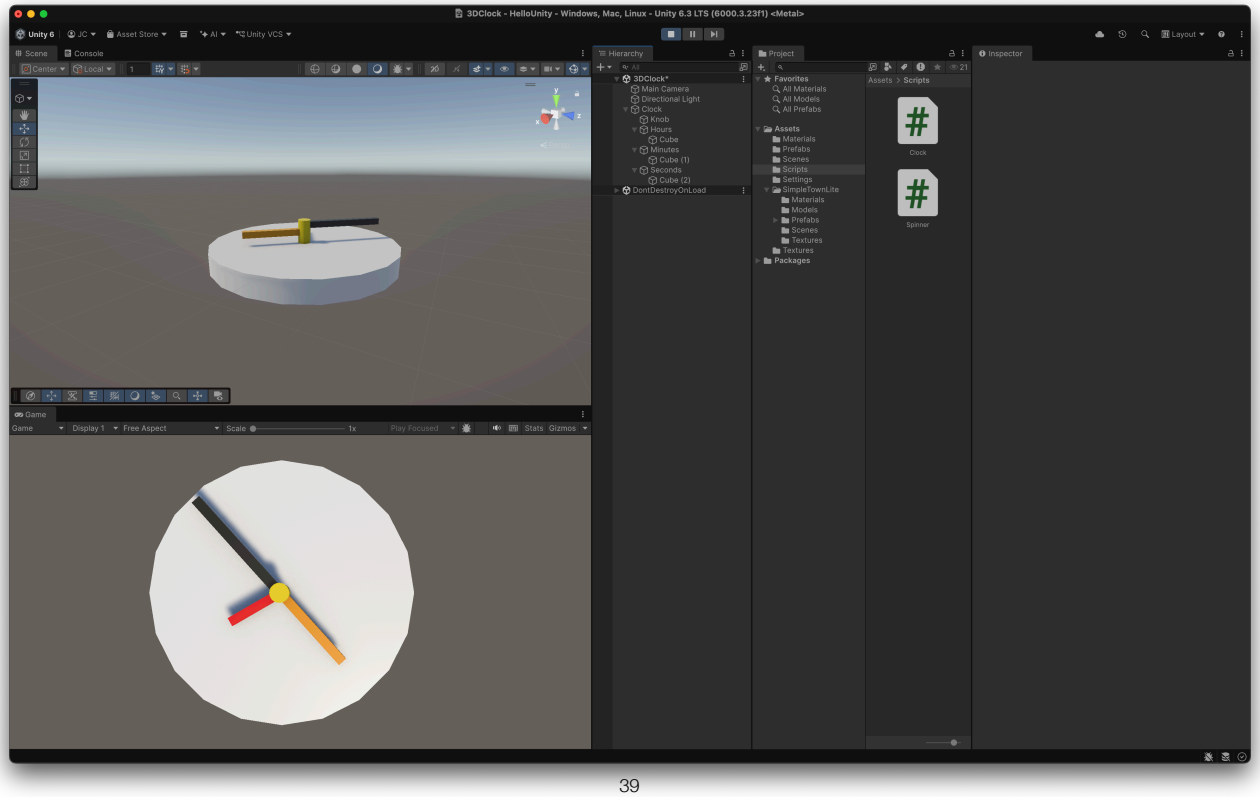


Building a simple 3D clock

12. Save the script and return to Unity.
13. Select the **Clock** object and drag and drop the **Clock** script on it.
14. You should now see three fields for assigning **Transform** components. Drag and drop **Hours**, **Minutes**, and **Seconds** GameObjects into their corresponding fields.
15. Click **Play** and see how the clock hands move according to your computer's current time.

Building a simple 3D clock





Conclusion

- We were introduced to C# programming with Unity and learned how C# scripts work with other Unity components.
- In the next chapter:
  - We will explore C# programming with Unity in more depth and learn how to create components that communicate with one another to build a complete interactive application.

Key points

- C# scripts can inherit from the **MonoBehaviour** class, allowing them to be used as components in Unity.
- A class that inherits from the **MonoBehaviour** can use special methods such as **Start**, **Update**, **Awake**, and **FixedUpdate**, which are called by Unity at specific times.
- A script can use the **GetComponent** method to access other components and modify their properties to create game logic and implement interactions.

Augmented Reality: Window To The World I The Road to Autonomy